

VZ200 VZ300

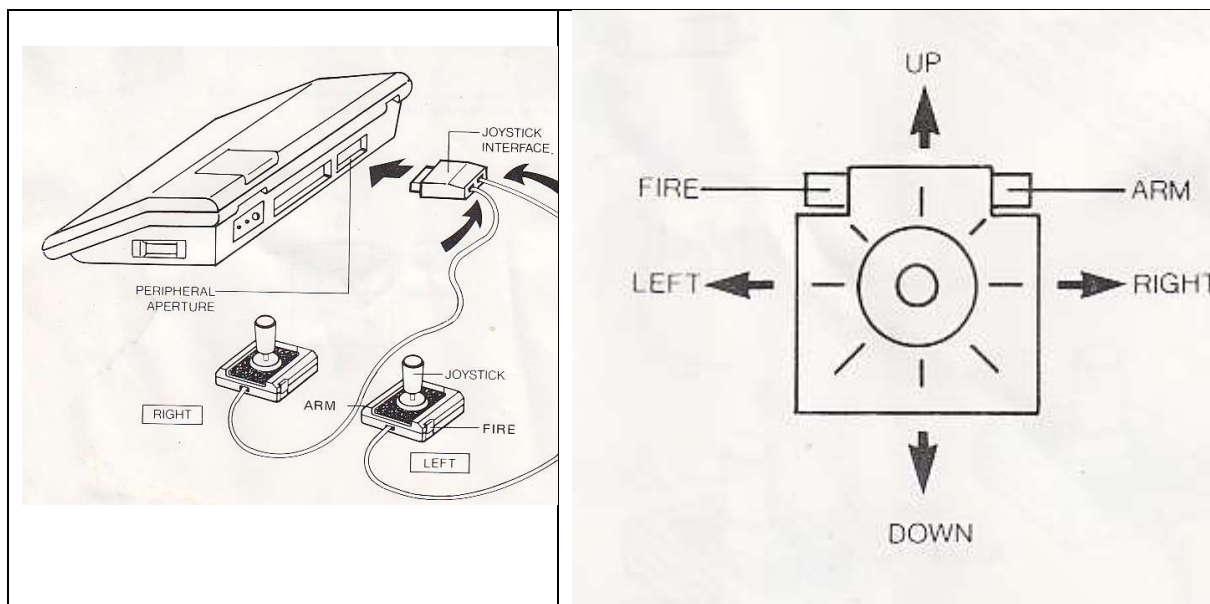
Wireless Bluetooth Joysticks



This project documents the design and construction of a replacement joystick module for the VZ/Laser computers. This could be a useful project for users wanting to upgrade their original VTech joysticks, or users unable to obtain the vintage joysticks and are willing to build one.

History

The original VZ joystick interface consisted of a plug in module connected to two joysticks.

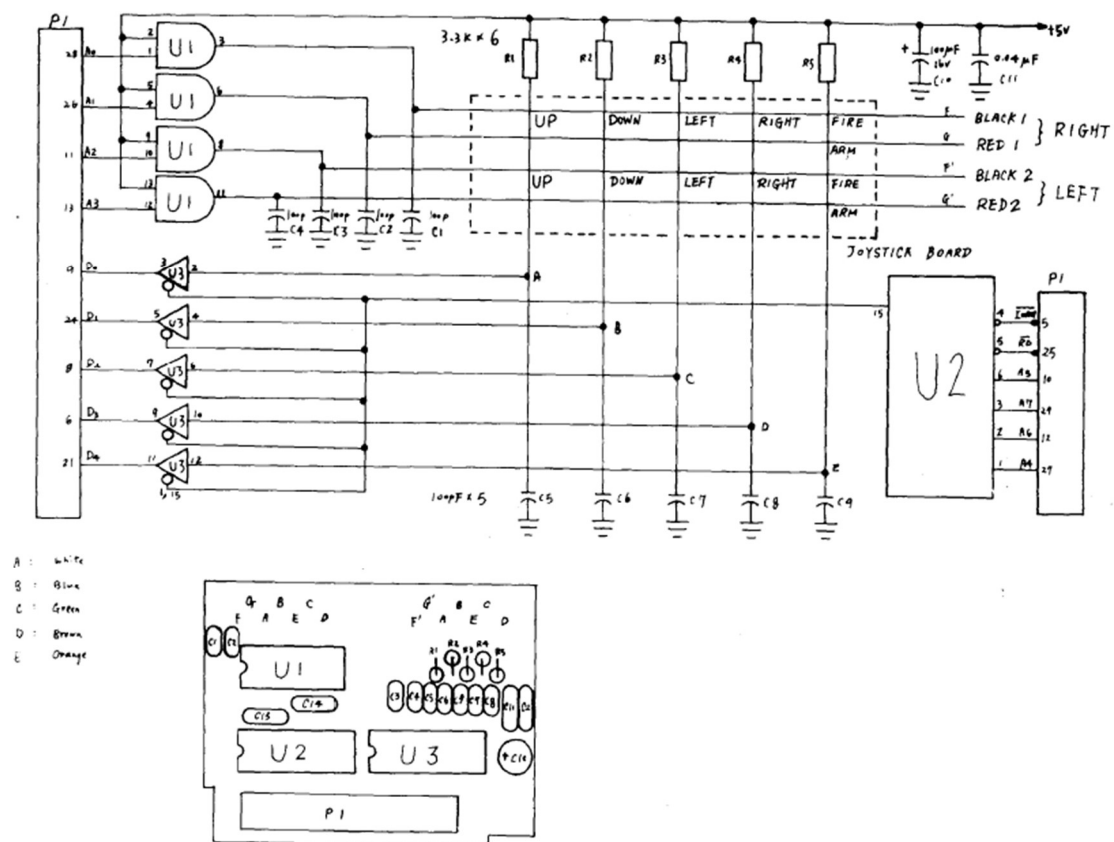


Each joystick could be moved in 8 directions. There were two independent buttons, labelled “fire” and “arm”

Under the hood

Each joystick contains 4 switches for the joystick movement (left, right, up, down) plus 2 switches for the buttons. Moving the joystick or pressing a button closes a switch.

The original circuit diagram is shown below:



The circuit has three IC's:

- U1 – 74LS08 AND gate
- U2 – 74LS138 Address decoder
- U3 – 74LS367 Buffer

Operation

Address lines A0 to A3 select the joystick/button combo:

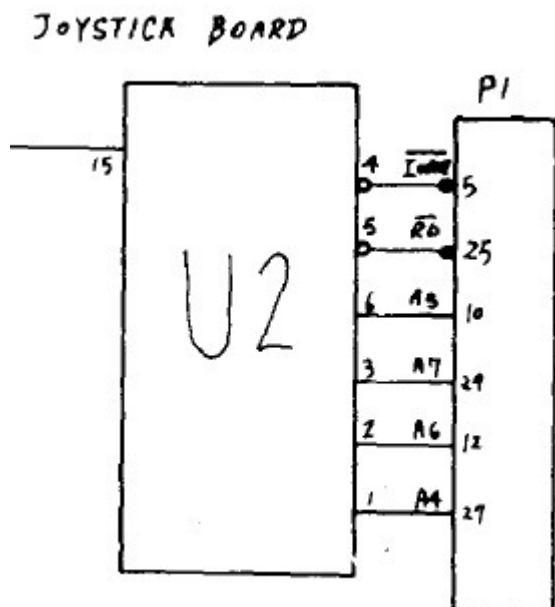
- A0 selects the right joystick and “fire” button
- A1 selects the right “arm” button
- A2 selects the left joystick and “fire” button
- A3 selects the left “arm” button

Each address line forms one input to an AND gate. The other input is tied high by a matrix of 5 x 3.3k pullup resistors connected to the joystick switches. Closing a switch takes the line to ground, giving a logic low. The combined value of this 5 bit matrix is read back by data lines D0-D4.

Using the right joystick as an example, if the joystick was idle D0-D4 would contain 001-11111 (63 decimal) on port 00101110 (port 46 decimal) The bit pattern for D7-D5 will always be 001.

If the joystick was pushed up and left, this would close switches connected to D0 and D2, returning 001-11010 (58 decimal)

A 74LS138 address decoder (U2) is connected to IORQ, RD, A4, A5, A6, A7 as below



A logic high on A5 with the other signals low triggers an active low signal on Y0 (pin 15)

This corresponds to an I/O read operation on ports 20h-2Fh (32 to 47 decimal)

Y0 (pin 15) is connected to the output enable pins (1 & 15) of the SN74LS367 buffer. This allows the joystick data to be placed on the Z80 data bus during the IORQ read operation. For all other states the joystick data is isolated from the data bus.

The table below illustrates the address/data mapping. A particular switch is selected by setting its address line to active zero during the I/O read.

	Address In								State	Data Out							
	A 7	A 6	A 5	A 4	A 3	A 2	A 1	A 0		D 7	D 6	D 5	D 4	D 3	D 2	D 1	D 0
Right	0	0	1	0	1	1	1	0	CENTER	0	0	1	1	1	1	1	1
	0	0	1	0	1	1	1	0	UP	0	0	1	1	1	1	1	0
	0	0	1	0	1	1	1	0	DOWN	0	0	1	1	1	1	0	1
	0	0	1	0	1	1	1	0	LEFT	0	0	1	1	1	0	1	1
	0	0	1	0	1	1	1	0	RIGHT	0	0	1	1	0	1	1	1
	0	0	1	0	1	1	1	0	FIRE	0	0	1	0	1	1	1	1
	0	0	1	0	1	1	0	1	ARM	0	0	1	0	1	1	1	1

	Address In								State	Data Out							
	A 7	A 6	A 5	A 4	A 3	A 2	A 1	A 0		D 7	D 6	D 5	D 4	D 3	D 2	D 1	D 0
Left	0	0	1	0	1	0	1	1	CENTER	0	0	1	1	1	1	1	1
	0	0	1	0	1	0	1	1	UP	0	0	1	1	1	1	1	0
	0	0	1	0	1	0	1	1	DOWN	0	0	1	1	1	1	0	1
	0	0	1	0	1	0	1	1	LEFT	0	0	1	1	1	0	1	1
	0	0	1	0	1	0	1	1	RIGHT	0	0	1	1	0	1	1	1
	0	0	1	0	1	0	1	1	FIRE	0	0	1	0	1	1	1	1
	0	0	1	0	0	1	1	1	ARM	0	0	1	0	1	1	1	1

The following test program was included in the original joystick instruction manual

By checking the value of A, you can determine the status of the left or right Joystick. For example, the following BASIC program will check the left, right, up and down directions of the LEFT joystick.

```

Example:  10 CLS
          20 A = (INP (43) AND 31)
          30 IF A = 30 THEN PRINT "UP": GO TO 20
          40 IF A = 29 THEN PRINT "DOWN": GO TO 20
          50 IF A = 27 THEN PRINT "LEFT": GO TO 20
          60 IF A = 23 THEN PRINT "RIGHT"
          70 GO TO 20
  
```

When this program is running, you can move the LEFT Joystick in any direction, and that direction will appear on screen.

To stop this program, press the **CTRL** and **[]** keys at the same time to get **BREAK**

Fast forward 40 years, and I'm still playing with my VZ.

But now I want to use my modern wireless game controller to play VZ-Panik and Dawn Patrol. So a module is required that translates events from my new game controller into a bit stream that can be read back into the VZ.

The brains of the operation is the "ESP32 WROOM DEV BOARD". It is a dual core microcontroller running up to 240mhz, with built-in WIFI and Bluetooth. It is a popular chip in the "Internet of Things" maker community. It is also 5v tolerant on the GPIO pins, which allows it to accept signals from Z80 era IC's without having to use level shifters.

As of 2024, it costs around \$13 AUD.



To reduce power consumption, the ESP32 will be programmed to operate at its lowest Bluetooth enabled speed – 80mhz. This is still plenty fast enough to control a joystick.

Design

The ESP32 will be powered directly from the VZ's 5 volt line. On startup the ESP32 will allow pairing of a new Bluetooth game controller, or connect automatically with a previously paired controller.

Once paired, the ESP32 will continually scan the game controller hundreds of times per second via the Bluetooth connection. The game controller events (stick movement & button presses) will be translated into the VZ joystick bit pattern. On a joystick port read from the VZ, the bits will be released to the data bus.

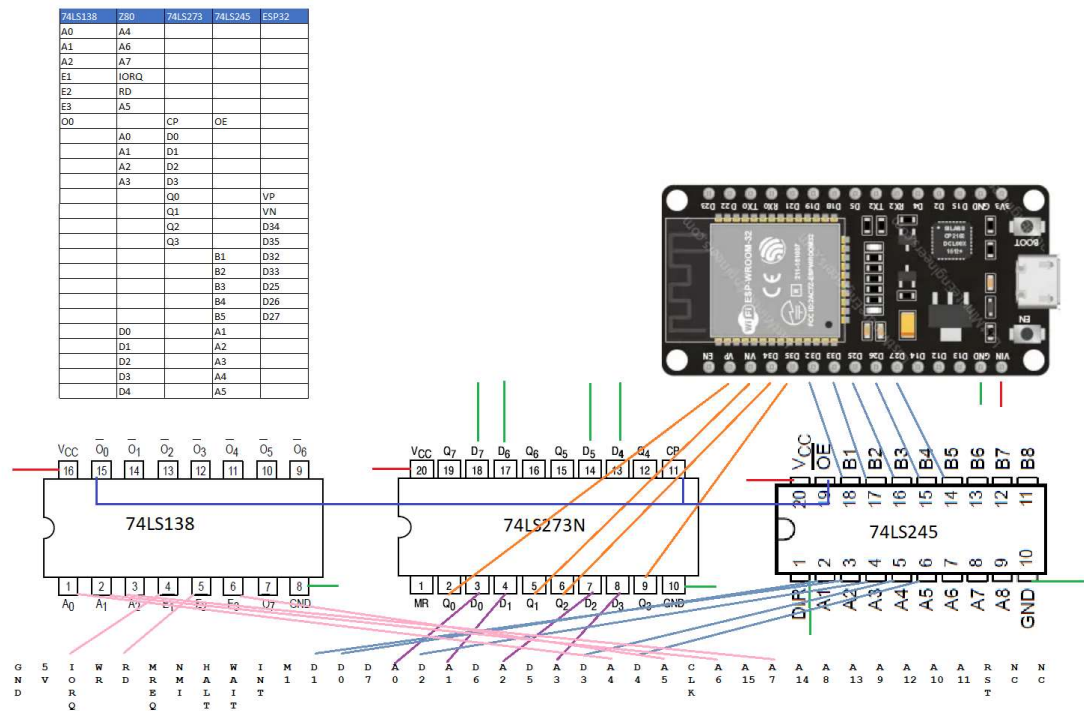
Three additional ICs are required to facilitate this:

1. 74LS138 address decoder. As with the original joystick controller, an address decoder is required to detect the IORQ + RD operation on the required port
2. 74LS273 latch. Used to sample joystick address lines A0-A3 on the IORQ + RD operation

- 74LS245 buffer. Used to isolate the joystick data from the Z80 data bus until the IORQ + RD operation. The 74LS367 used in the original joystick module could also be used. I chose the 74LS245 as I have a number of these in my junkbox

Wiring

The circuit is easy to wire up on a breadboard for testing



- 104uf capacitors are added for each of the 74LS IC's between 5v and ground
- Unused inputs on the LS273 are tied to ground (D7, D6, D5, D4)



An "8BitDo SN30" gamepad was used for testing. It allows Bluetooth and wired USB connection

There was no observable latency. It functions exactly like a VZ joystick

Cost is around \$60 AUD

The sketch

```
#include <Bluepad32.h>
```

```
const int Z80_D0 = 32;
```

```
const int Z80_D1 = 33;
```

```
const int Z80_D2 = 25;
```

```
const int Z80_D3 = 26;
```

```
const int Z80_D4 = 27;
```

```
const int Z80_D5 = 14;
```

```
const int Z80_A0 = 36;
```

```
const int Z80_A1 = 39;
```

```
const int Z80_A2 = 34;
```

```
const int Z80_A3 = 35;
```

```
byte      A00;
```

```
byte      A01;
```

```
byte      A02;
```

```
byte      A03;
```

```
byte      VZJoystickByte;
```

```
ControllerPtr myControllers[BP32_MAX_GAMEPADS];
```

```
// This callback gets called any time a new gamepad is connected.
```

```
// Up to 4 gamepads can be connected at the same time.
```

```
void onConnectedController(ControllerPtr ctl) {
```

```
    bool foundEmptySlot = false;
```

```
    for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
```

```
        if (myControllers[i] == nullptr) {
```

```
            // Additionally, you can get certain gamepad properties like:
```

```

        // Model, VID, PID, BtAddr, flags, etc.
        ControllerProperties properties = ctl->getProperties();
        myControllers[i] = ctl;
        foundEmptySlot = true;
        break;
    }
}

}

void onDisconnectedController(ControllerPtr ctl) {
    bool foundController = false;

    for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
        if (myControllers[i] == ctl) {
            myControllers[i] = nullptr;
            foundController = true;
            break;
        }
    }
}

void processGamepad(ControllerPtr ctl) {

    if (ctl->dpad() & 0x01) VZJoystickByte &= 0x3E;    // up
    if (ctl->dpad() & 0x02) VZJoystickByte &= 0x3D;    // down
    if (ctl->dpad() & 0x04) VZJoystickByte &= 0x37;    // right
    if (ctl->dpad() & 0x08) VZJoystickByte &= 0x3B;    // left

    if ((ctl->buttons() & 0x0001) && (A01 == LOW)) VZJoystickByte = 0x2F;
// arm      button B

    if ((ctl->buttons() & 0x0004) && (A00 == LOW)) VZJoystickByte &= 0x2F;
// fire      button Y
}

```

```

// Arduino setup funcion. Runs in CPU 1
void setup() {

    pinMode(Z80_D5, OUTPUT);
    pinMode(Z80_D4, OUTPUT);
    pinMode(Z80_D3, OUTPUT);
    pinMode(Z80_D2, OUTPUT);
    pinMode(Z80_D1, OUTPUT);
    pinMode(Z80_D0, OUTPUT);

    pinMode(Z80_A0, INPUT);
    pinMode(Z80_A1, INPUT);
    pinMode(Z80_A2, INPUT);
    pinMode(Z80_A3, INPUT);

    const uint8_t* addr = BP32.localBdAddress();

    // Setup the Bluepad32 callbacks
    BP32.setup(&onConnectedController, &onDisconnectedController);

    // "forgetBluetoothKeys()" should be called when the user performs
    // a "device factory reset", or similar.
    // Calling "forgetBluetoothKeys" in setup() just as an example.
    // Forgetting Bluetooth keys prevents "paired" gamepads to reconnect.
    // But might also fix some connection / re-connection issues.
    BP32.forgetBluetoothKeys();

    // Enables mouse / touchpad support for gamepads that support them.
    // When enabled controllers like DualSense and DualShock4 generate two
    connected devices:
    // - First one: the gamepad
    // - Second one, which is a "vritual device", is a mouse
    // By default it is disabled.
    //BP32.enableVirtualDevice(false);
}

```

```

// Arduino loop function. Runs in CPU 1
void loop() {

    VZJoystickByte = 0x3F;    // no buttons pressed, return byte is 63
    decimal

    A00 = digitalRead(Z80_A0);
    A01 = digitalRead(Z80_A1);
    A02 = digitalRead(Z80_A2);
    A03 = digitalRead(Z80_A3);

    // This call fetches all the gamepad info from the NINA (ESP32) module.
    // Just call this function in your main loop.
    // The gamepads pointer (the ones received in the callbacks) gets
    updated
    // automatically.
    BP32.update();

    // It is safe to always do this before using the gamepad API.
    // This guarantees that the gamepad is valid and connected.
    for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
        ControllerPtr myController = myControllers[i];

        if (myController && myController->isConnected()) {
            if (myController->isGamepad()) {
                processGamepad(myController);
            } else {
                continue;
            }
        }
    }
}

```

```
digitalWrite(Z80_D5, (VZJoystickByte & 0x20));  
digitalWrite(Z80_D4, (VZJoystickByte & 0x10));  
digitalWrite(Z80_D3, (VZJoystickByte & 0x08));  
digitalWrite(Z80_D2, (VZJoystickByte & 0x04));  
digitalWrite(Z80_D1, (VZJoystickByte & 0x02));  
digitalWrite(Z80_D0, (VZJoystickByte & 0x01));  
  
// The main loop must have some kind of "yield to lower priority task"  
event.  
  
delay(5);  
}
```

Possible enhancements

1. Add a switch to allow selection between left & right joystick mapping
2. Add connectors for a nintendo gamepad/Atari joystick/custom arcade joystick

